Appendix A

HOMOGENEOUS COORDINATES

A.1 DEFINITION

Many operations of three-dimensional graphics and geometry are most compactly represented in terms of so-called *homogeneous coordinates*. Homogeneous coordinates represent three-dimensional points in terms of four-dimensional ones by adding one additional coordinate axis called w. Hence, the three-dimensional point $(x\ y\ z)$ is represented by a four-dimensional point $(x'\ y'\ z'\ w)$ such that

$$\begin{aligned} x &= x'/w \\ y &= y'/w \\ z &= z'/w \end{aligned} \tag{A.1}$$

The homogeneous coordinate representation offers many advantages. For instance, the four-dimensional representation can be scaled at will by multiplying all coordinates by the same factor, as this will not affect the correspondence given by the Equations A.1. Observe also that points at infinity can be conveniently represented by choosing $w = 0$.

A.2 COORDINATE TRANSFORMATIONS

By far the most important property of homogeneous coordinates is that they allow all *coordinate transformations* needed in viewing and modeling to be represented with 4-by-4 matrices.

A.2.1 Translation

The *translation transformation* that transforms a point $(x\ y\ z)$ to

$$(x + t_x \quad y + t_y \quad z + t_z)$$

is represented by the matrix T constructed as follows:

$$T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \tag{A.2}$$

Given the matrix T, the translation can be calculated in the homogeneous coordinate representation as the vector-matrix product

$$[x'\ y'\ z'\ w] \times T = [x' + t_x w \quad y' + t_y w \quad z' + t_z w \quad w]^T.$$

The inverse translation is simply

$$T(-t_x, -t_y, -t_z) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -t_x & -t_y & -t_z & 1 \end{bmatrix} \tag{A.3}$$

A.2.2 Rotation

The *rotation transformation* that rotates points around one of the coordinate axes has a similar simple structure. The rotation of ϕ degrees around the z-axis is represented by the matrix R_z as follows:

$$R_z(\phi) = \begin{bmatrix} \cos\phi & \sin\phi & 0 & 0 \\ -\sin\phi & \cos\phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.4}$$

Angle ϕ represents a counterclockwise rotation; for instance, if $\phi = 90$ degrees, $R_z(\phi)$ maps the point $(1\ 0\ 0\ 1)$ to the point $(0\ 1\ 0\ 1)$. Rotation matrices R_x and R_y that represent corresponding rotations around x- and y-axes are given by the following similar matrices:

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\phi & \sin\phi & 0 \\ 0 & -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.5}$$

$$R_y(\phi) = \begin{bmatrix} \cos\phi & 0 & -\sin\phi & 0 \\ 0 & 1 & 0 & 0 \\ \sin\phi & 0 & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.6}$$

A.2.3 Scaling

The *scaling transformation* $S(s_x, s_y, s_z)$ can also be expressed as a 4-by-4 matrix:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.7}$$

A.3 COMBINATION OF TRANSFORMATIONS

A major advantage of homogeneous coordinate transformations is that they can be piled upon each other. Specifically, an arbitrary series of translations and rotations can be combined into a single transformation matrix simply by calculating the product of the corresponding translation or rotation matrices in the correct order. For instance, the composite transformation matrix M consisting of a translation $T = T(t_x, t_y, t_z)$ followed by rotations $R_1 = R_x(\phi_1)$ and $R_2 = R_x(\phi_2)$ can be produced as the matrix product

$$M = T \times R_1 \times R_2.$$

Combined, an arbitrary sequence of rotations will yield a general rotation matrix R of the form

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.8}$$

A useful observation is the fact that the 3-by-3 submatrix $[r_{ij}]$ is *orthogonal*, i.e., its column vectors

$$v_i = [r_{1i}\ r_{2i}\ r_{3i}]^T, i = 1, 2, 3$$

are mutually orthogonal unit vectors. Under R, they will be mapped to unit vectors on x-, y-, and z-axes, respectively.

Another useful property of an orthogonal matrix is that its inverse is simply its transpose. Hence the inverse transformation matrix R^{-1} of R is simply

$$R^{-1} = R^T = \begin{bmatrix} r_{11} & r_{21} & r_{31} & 0 \\ r_{12} & r_{22} & r_{32} & 0 \\ r_{13} & r_{23} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{A.9}$$

A.4 A VIEWING OPERATION

As a useful example of the homogeneous coordinates, let us consider the problem of performing a *3-dimensional viewing operation*, i.e., generating a two-dimensional projected figure of a three-dimensional object such that could be used to drive a graphical display or a plotter. Again, textbooks on computer graphics should be consulted for the full account of the subject. Here we restrict ourselves to just one particular solution.

A.4.1 Specification of the Viewing Operation

We specify the viewing operation in terms of two points,

$$eye = (e_x\ e_y\ e_z)$$

and

$$ref = (r_x\ r_y\ r_z)$$

and a vector

$$up = [u_x\ u_y\ u_z].$$

Points *eye* and *ref* constitute the elements of the so-called *synthetic camera analogy* popular in three-dimensional computer graphics. In particular, *eye* gives the location of the camera, and *ref* the location the camera is directed at. Hence the vector

$$dir = eye - ref = [e_x - r_x\ \ e_y - r_y\ \ e_z - r_z]$$

gives the direction vector of the camera from *ref*. For instance, $dir = [0\ 0\ 1]$ corresponds with a camera viewing a "top" view of the scene.

Recall that a vector and a point uniquely define a plane that passes through the point with the normal given by the vector. In our case, *dir* and *ref* define the *viewing plane* onto which we shall project our scene. In the synthetic camera analogy, the viewing plane corresponds with the film plane onto which the image is generated.

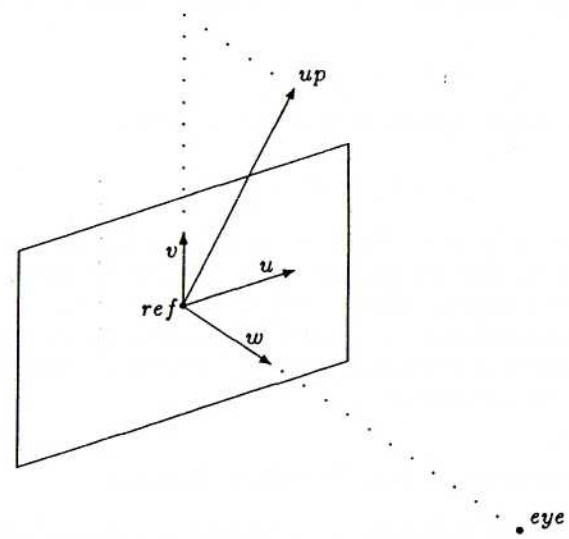

Figure A.1 Three-dimensional viewing operation.

The coordinates u and v will be used on the viewing plane. To make the viewing operation a coordinate transformation from the xyz-space to another three-dimensional space, it is useful to introduce also a third coordinate w that gives "depth" of a point with respect to the viewing plane.[1] To establish the transformation needed, let us fix the origin of the uvw-space at the reference point ref. Then dir establishes the direction vector of w-axis in terms of x, y, and z. What remains is the fixing of u- and v-axes with respect to the xyz-space—our synthetic camera points into correct direction, but it may still rotate around the axis perpendicular to the film plane.

The specification is completed with the third piece of input, the vector up. Its components $[u_x\ u_y\ u_z]$ give a direction of xyz-space that should appear "upwards" (i.e., along the v-axis) in the figure. Hence, in a "top" view,

$$[u_x\ u_y\ u_z] = [0\ 1\ 0]$$

makes the y-axis to appear upwards in the figure. Likewise,

$$[u_x\ u_y\ u_z] = [0\ 0\ 1]$$

makes the z-axis appear upwards. This effect is accomplished by selecting the projection of up on the viewing plane as the direction of the v-axis. Obviously, up must not be perpendicular with the viewing plane.

After the w- and v-axes have been fixed, the u-axis is simply formed by a vector (cross) product of v and w. The specification of the viewing operation is illustrated in Figure A.1.

A.4.2 Construction of the Viewing Matrix

The viewing operation can be compactly represented as a *viewing matrix*, constructed as follows:

1. Create a translation matrix t, effecting the change of origin to point ref. The resulting $x'y'z'$-space now has the same origin the uvw-space.

2. Construct a rotation matrix r that represents the coordinate rotation from the $x'y'z'$-space to the uvw-space.

3. Matrix multiply t and r to yield the desired result, matrix $view$.

[1] The depth information is needed for hidden line and hidden surface removal as well.

By using the special properties of orthogonal rotation matrices, both $view$ and its inverse $view^{-1}$ can be constructed directly (i.e., without explicit matrix inversion). Let

$$\begin{aligned}
[u] &= [u_{x'}\ u_{y'}\ u_{z'}]^T \\
[v] &= [v_{x'}\ v_{y'}\ v_{z'}]^T \\
[w] &= [w_{x'}\ w_{y'}\ w_{z'}]^T
\end{aligned}$$

be the unit vectors along the u-, v-, and w-axes in terms of x', y', and z'. Then the rotation from $x'y'z'$ to uvw is given by the following matrix:

$$\begin{bmatrix} [u] & [v] & [w] & 0 \\ & 0 & & 1 \end{bmatrix}$$

Because u, v, and w are orthogonal unit vectors, the inverse transformation is given simply by the transpose matrix:

$$\begin{bmatrix} [\ u\ ]^T & & \\ [\ v\ ]^T & & 0 \\ [\ w\ ]^T & & \\ 0 & & 1 \end{bmatrix}$$

Hence the derivation of the viewing matrix reduces to forming the unit vectors along u, v, and w. A procedure that performs this operation is given as Program A.1. It uses the vector and matrix algebra package described in Section A.5.

A.5 A VECTOR AND MATRIX PACKAGE

This section introduces a collection of simple procedures for processing transformation matrices and doing certain other basic operations for vectors. To simplify the notation, we shall use the C type definitions

```
typedef float vector[4];
typedef float matrix[4][4];
```

```
void    makeview(eye, ref, up, view, viewinv)
vector  eye, ref, up;
matrix  view, viewinv;
{
        matrix          t, r, ti, ri;
        vector          u, v, w;
        double          d;
        int             i;

        w[0] = eye[0] - ref[0];
        w[1] = eye[1] - ref[1];
        w[2] = eye[2] - ref[2];
        normalize(w);
        d = dot(up, w);
        v[0] = up[0] - d*w[0];
        v[1] = up[1] - d*w[1];
        v[2] = up[2] - d*w[2];
        normalize(v);
        cross(u, v, w);
        matident(t);
        mattrans(t, -ref[0], -ref[1], -ref[2]);
        matident(r);
        for(i=0; i<3; i++)
        {
                r[i][0] = u[i];
                r[i][1] = v[i];
                r[i][2] = w[i];
        }
        matmult(view, t, r);
        matident(ti);
        mattrans(ti, ref[0], ref[1], ref[2]);
        matident(ri);
        for(i=0; i<3; i++)
        {
                ri[0][i] = u[i];
                ri[1][i] = v[i];
                ri[2][i] = w[i];
        }
        matmult(viewinv, ri, ti);
}
```

Program A.1 Construction of the viewing matrix.

that define a 4-vector and a 4-by-4 matrix. These definitions are also used in the main text.

```
matident(m)
matrix m;
```

Make m an identity matrix.

```
mattrans(m, tx, ty, tz)
matrix m;
float tx, ty, tz;
```

Combine a translation ($tx\ ty\ tz$) with m by constructing a translation matrix t from tx, ty, and tz and multiplying m with t on the right.

```
matrotat(m, rx, ry, rz)
matrix m;
float rx, ry, rz;
```

Combine a rotation ($rx\ ry\ rz$) with m by constructing a rotation matrix r from tx, ty and tz and multiplying m with r on the right. If several rotations are specified, they are applied in the order rx, ry, rz.

```
matmult(m, m1, m2)
matrix m, m1, m2;
```

Calculate the matrix product $m = m1 \times m2$. The procedure should be implemented so that the argument matrix for the result m can be same as $m1$ or $m2$.

```
vecmult(v1, v2, m)
vector v1, v2;
matrix m;
```

Calculate the vector-matrix product $v1 = v2 \times m$. The procedure should be implemented so that $v1$ can equal $v2$.

The following procedures only work on the three first components of their argument vectors.

```
double dot(v1, v2)
vector v1, v2;
```

Calculate the dot (scalar) product of $v1$ and $v2$.

```
cross(v1, v2, v3)
vector v1, v2, v3;
```

Calculate the cross (vector) product $v1 = v2 \times v3$. The procedure should be implemented so that the argument vector for the result *v1* can be same as *v2* or *v3*.

```
double normalize(v)
vector v;
```

Make *v* a unit vector, return its previous length.

```
veccopy(v1, v2)
vector v1, v2;
```

Make *v1* equal to *v2*.

```
vecplus(res, v1, v2)
vector res, v1, v2;
```

Sum vectors *v1* and *v2* into *res*.

```
vecminus(res, v1, v2)
vector res, v1, v2;
```

Let $res = v1 - v2$.

```
vecnull(v, tol)
vector v;
double tol;
```

Test whether *v* is a null vector up to tolerance *tol*.

```
vecequal(v1, v2)
vector v1, v2;
```

Check whether vectors *v1* and *v2* are the same.